

TINIEE: Traffic-Aware Adaptive In-Network Intelligence via Early-Exit Strategy

Heewon Kim[✉], *Member, IEEE*, Seongyeon Yoon, *Member, IEEE*, Chanbin Bae[✉], *Member, IEEE*,
Sanghoon Lee, *Member, IEEE*, and Sangheon Pack[✉], *Senior Member, IEEE*

Abstract—In-network inference over programmable data planes (PDPs) allows fast and low-overhead inference using deep neural networks (DNNs). To alleviate massive processing and deployment costs of in-network inference, distributed deployment on multiple programmable network devices is mainly adopted. However, it is likely to produce a considerable amount of network traffic due to the long distance of the forwarding path and the intermediate data between submodels. In this work, we propose a traffic-aware adaptive in-network inference scheme, TINIEE, to significantly reduce the network traffic of in-network inference without causing a considerable reduction in classification performance. To this end, we first devise an adaptive inference method on the data plane that strikes the balance between the classification performance and the network traffic cost. Furthermore, we formulate a traffic minimization problem to decide a proper location of each submodel considering each flow's exit tendency with a predefined confidence threshold. Since the problem is excessively complicated, we devise a low-complexity practical submodel placement algorithm. We implement TINIEE on both software and hardware programmable switches, and the evaluation results demonstrate that TINIEE reduces network traffic by up to 34.48%, maximum link utilization up to 52.7%, and flow completion time up to 17.8% compared to the state-of-the-art, while maintaining a high classification performance.

Index Terms—Adaptive inference, early-exit, in-network intelligence, programmable data plane, P4.

I. INTRODUCTION

SOFTWARE-DEFINED networking (SDN) is being adopted in various types of networks (e.g., mobile and data center networks), which separates the control plane and the data plane to make networks more scalable and flexible. This paradigm provides an opportunity for full programmability of the data plane (i.e.,

programmable data plane (PDP)) as well as the control plane, enabling customized packet processing so that a new protocol or operation can be applied during packet forwarding [1].

Currently, the research community has been dedicated to utilizing PDP to offload operations (e.g., aggregation [2], monitoring [3], and caching [4]) that are commonly performed on a distinct middlebox or server. In this context, in-network intelligence (INI) is one of the most promising PDP techniques that implement intelligent functionalities (e.g., traffic classification) on the data plane [11]. INI provides a definite advantage in alleviating network overhead (e.g., end-to-end latency) over traditional server-based approaches, as the intelligent function can be carried out on the forwarding nodes [5] without visiting other computing nodes. Furthermore, it also has benefits from a security perspective, as there is no information leakage on the server [10].

Meanwhile, as an extension of applying machine learning to in-network intelligence [6], [7], researchers are investigating how to use deep neural networks (DNN) directly due to their powerful performance and adjustability [8], [9], [10], [11], [12], [13], [14], [17], [18], [19]. However, to implement DNN in PDP, several challenges need to be overcome. First, current PDP architectures (e.g., PISA [28]) do not support floating-point operations, which are the basis for the forward and back-propagation of DNN. To address this issue, existing work employs a binary neural network (BNN) [12] or knowledge distillation [11] to convert the model into one compatible with PDP. Second, it is necessary to control the high processing costs and massive memory consumption that a PDP-capable node should bear as the deployed model grows larger. To address this issue, splitting a DNN model into several submodels and deploying them on multiple nodes is one feasible option [17], [18], [19].

However, simply splitting the model does not effectively reduce the overall processing cost [10]; it brings an additional issue of how to orchestrate multiple submodels in the network. This is because each flow should pass through nodes sequentially with the correct order of submodels to be inferred correctly (i.e., exclusive forwarding chain). Under these circumstances, without careful deployment of submodels, the flow can traverse a long distance compared to its shortest path, producing massive traffic in the network. Moreover, the existence of intermediate data between submodels aggravates this problem, since it should be delivered to the following submodel for the consecutive inference process. Consequently, to make the best use of in-network inference without concern for network problems such as congestion, it is crucial to carefully orchestrate a network-wide in-network inference

Received 24 December 2024; revised 10 March 2026; accepted 24 April 2026; approved by IEEE TRANSACTIONS ON NETWORKING Editor S. Lin. Date of publication 1 May 2026; date of current version 11 May 2026. This work was supported in part by the National Research Foundation (NRF) funded by Korean Government [Minister of Science and ICT (MSIT)] under Grant RS-2026-25499410, Grant RS-2022-NR070834, and Grant RS-2024-00341965; and in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP)-Information Technology Research Center (ITRC) under Grant IITP-2026-RS-2021-II211810. A preliminary version has been presented at IEEE International Conference on Sensing, Communication, and Networking (SECON) 2024 [DOI: 10.1109/SECON64284.2024.10934959]. (*Corresponding author: Sangheon Pack.*)

Heewon Kim, Chanbin Bae, and Sangheon Pack are with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea (e-mail: harry0475@korea.ac.kr; bin6050@korea.ac.kr; shpack@korea.ac.kr).

Seongyeon Yoon is with the Network Technology and Operation Group, LG Uplus Corporation, Seoul 04389, South Korea (e-mail: ysy0326@korea.ac.kr).

Sanghoon Lee is with Core Network Group, SK Telecom, Seoul 04539, South Korea (e-mail: shunlee134@korea.ac.kr).

Digital Object Identifier 10.1109/TON.2026.3689743

procedure while adopting an adaptive mechanism that can control network traffic during the inference.

In this paper, we present a traffic-aware adaptive in-network inference scheme called TINIEE to reduce the traffic cost of in-network inference while guaranteeing high classification performance. At first, we devise an adaptive in-network inference method on the data plane based on a widely used adaptive inference method, early-exit [23]. To obtain an appropriate confidence score during packet processing, we use a match-action table where possible value entries are pre-calculated. In addition, to compress the table size, we propose a rule generation method that controls the number of rules by managing the granularity of the confidence value, called the GRACE algorithm. Furthermore, we propose a network-level submodel orchestration method on the control plane. For this, we formulate a traffic minimization problem for proper submodel placement, considering a predefined confidence threshold as a mixed-integer linear programming (MILP) problem, which is proven to be NP-hard. To obtain a practical solution, we devise an early-exit-aware submodel placement (ESP) algorithm that can produce near-optimal solutions with low complexity. To verify the effectiveness of the TINIEE scheme, we implement TINIEE on both software and hardware programmable switches and check the clear controllability of the trade-off by means of the confidence threshold. Furthermore, the evaluation results in realistic environments demonstrate that TINIEE can reduce network traffic by up to 34.48%, maximum link utilization by up to 52.7%, and flow completion time by up to 17.8% compared to the state-of-the-art schemes (i.e., NNsplit [19] and DINC [43]), while maintaining comparable classification performance.

Our contributions can be summarized as follows.

- 1) To the best of our knowledge, TINIEE is the first adaptive in-network inference scheme that can effectively control the trade-off between network traffic and classification performance.
- 2) The GRACE algorithm can effectively reduce memory consumption on the data plane by decreasing the number of rules and the table size. We provide a detailed analysis of the impact of granularity.
- 3) The ESP algorithm is a low-complexity practical submodel orchestration algorithm that can be easily deployed in practical controllers with low overhead.
- 4) Extensive evaluation results are valuable as guidelines for the design of efficient distributed INI systems, which show that TINIEE simultaneously achieves the lowest traffic load and comparable classification performance.

The remainder of this paper is organized as follows. Section II describes the background and motivation. Section VI and Section III introduce the related work and the system model, respectively. The detailed description of TINIEE is provided in Section IV, and the evaluation results are presented in Section V. Finally, Section VII concludes this paper.

II. BACKGROUND AND MOTIVATION

A. Background

Programmable Data Plane. PDP allows customized packet forwarding, which can be realized by means of domain-specific high-level languages such as P4 [28], OpenState [29],

and Domino [30]. Most of all, P4 is the most widely used programming language, which specifies packet processing logic such as header parsing and applying match-action tables. Following the main goal of P4 to achieve target independence, P4 supports various target devices that cover a wide range of network elements (e.g., FPGA, NIC, hardware switch, and software switch) [1]. Here, each target device has its own limitations regarding performance and available resources. For example, a programmable hardware switch with a Tofino ASIC [27] shows high processing performance of up to tens of Tbps; however, it has a severe constraint on the number of pipeline stages to meet line-rate packet processing. On the other hand, FPGAs typically have lower throughput, showing 10-100 Gbps while having the advantage of greater flexibility. Note that in-network inference schemes directly implementing DNN utilize a wide range of target devices such as FPGAs [12], NICs [10], software switches [18], and hardware switches [13] to embody the operation of neural networks.

Early-Exit. Early-exit is an adaptive inference method used to evaluate the classification result in the middle of the inference and terminate the process if the result is sufficiently reliable [23]. To successfully manage the trade-off between accuracy and inference load, it is necessary to judge whether it is reasonable to exit the inference at a particular point. To this end, the existing work [23], [24] calculates the confidence score, which is the relative certainty of the output label compared to others. After that, the decision of whether to leave the inference is determined according to the exit policy; one common method is to set a static confidence threshold [25]. The architecture of the early-exit model is composed of a backbone network and early-exit branches. The backbone network is the original neural network. Meanwhile, the early exit branch is essentially a classification layer that produces the output value and optionally includes additional layers to improve performance. While training the early-exit model, multiple losses from exit points are reflected at once by conducting back-propagation with the sum of losses for each iteration.

B. Motivating Example

We describe our motivation using examples presented in Fig. 1, where the network traffic is visualized and generated in the leaf-spine network from source S to destination D . A traffic flow is expressed by node-to-node arrows, and the order of the traffic on the path is represented by the number in the middle of the arrow. Furthermore, traffic containing intermediate data is highlighted as a bold arrow. A DNN model is made up of three submodels M_1 , M_2 , and M_3 , and the classification layer is expressed as a block with C . The confidence scores when the flow exits on M_1 , M_2 , and M_3 are set to 0.75, 0.85, and 0.95, respectively. Lastly, the confidence threshold is assumed to be 0.8, which is regarded as a sufficient confidence value to guarantee accurate classification in this example.

Comparing Fig. 1(a) and Fig. 1(b), the positions of the three submodels are the same; however, static inference is applied in Fig. 1(a), while adaptive inference is applied in Fig. 1(b). Specifically, since the flow exits at M_2 in Fig. 1(b), the flow traverses a shorter path, leading to traffic reduction compared to Fig. 1(a) (from 8 to 6). Moreover, traffic generated by intermediate data is also decreased by 2/3 as traffic traversing

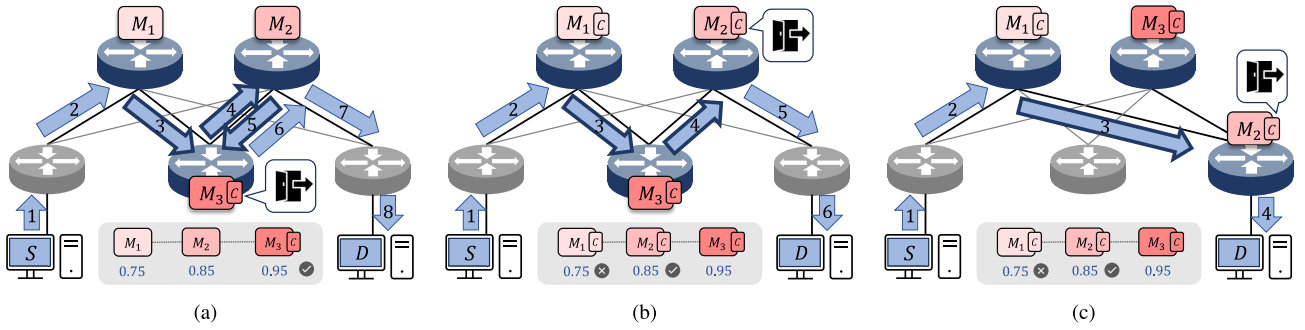


Fig. 1. The effect of in-network inference strategies. (a) Static in-network inference composed of three submodels, (b) Adaptive in-network inference without submodel orchestration, (c) Adaptive in-network inference with proper submodel orchestration.

between submodels is reduced. Meanwhile, in Fig. 1(c), the position of the submodels is changed compared to Fig. 1(b), while identical adaptive inference is applied. In this setting, it can be seen that the path length is even shorter, leaving much less traffic in the network (to 4). Furthermore, the intermediate data is far more reduced to 1/2 of that in Fig. 1(b).

To summarize, these examples reveal that the application of adaptive in-network inference can reduce overall network traffic; however, it also requires careful placement of submodels, considering where each flow exits, to take greater advantage of adaptive inference in the network.

III. SYSTEM MODEL

As shown in Fig. 2, we consider an SDN environment consisting of programmable network devices and a controller. On the control plane, the controller collects flow requests that contain traffic information such as source, destination, and flow size before an in-network inference process begins. At the same time, the controller splits a DNN model into submodels and composes an early-exit model by adding early-exit branches at the end of each submodel. The controller then trains the early-exit model and profiles it to check the confidence score tendency and the appropriate confidence threshold. With flow traffic information and the confidence threshold, the location of each submodel is determined using the ESP algorithm (Section IV-B), and the rules for confidence, routing tables, and weights are updated through APIs such as P4runtime [32]. The confidence table is used to obtain the confidence score in PDP, and its rules are generated by using the GRACE algorithm (Section IV-A).

After the preparation of the control plane, the data plane conducts adaptive in-network inference while simultaneously forwarding the packet. When a packet enters the network, it first passes through the inference devices sequentially. In each inference device, a part of the inference is applied with given weights, and the confidence score is evaluated for the intermediate result. If a packet does not satisfy the exit policy, it simply moves to the subsequent inference device following the inference routing table (① in Fig. 2) with the intermediate data contained in the packet header. Otherwise, it escapes from the inference path and directly heads to the destination following the normal routing table (② in Fig. 2). Note that the inference routing table and the normal routing table send packets toward consecutive inference devices and the destination, respectively.

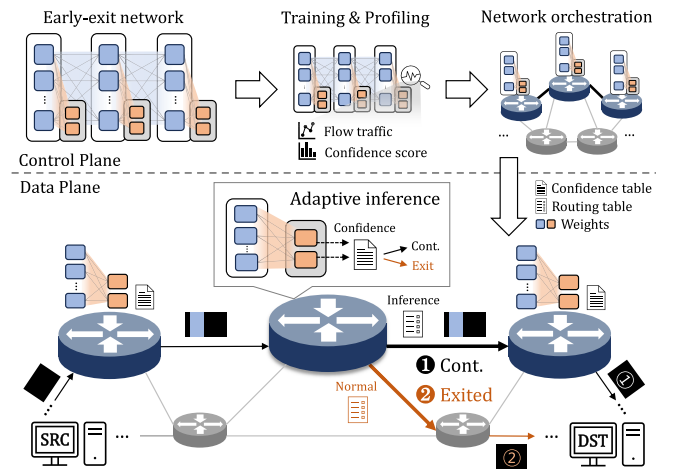


Fig. 2. Overview of TINIEE.

IV. TINIEE DESIGN

In this section, we describe the TINIEE design details, which can be distinguished as 1) an adaptive in-network inference procedure and 2) a network-level submodel placement algorithm. We first devise essential functional components (i.e., in-network inference, header, forwarding logic, and confidence estimation) to operate the adaptive in-network inference. In addition, to further maximize the effect of the adaptive mechanism, we formulate the traffic minimization problem and devise a practical heuristic algorithm to determine the proper placement of submodels considering the exit tendency.

A. Adaptive In-Network Inference

In-Network Inference Architecture. Fig. 3 shows the in-network inference architecture characterized by three components: 1) input building, 2) BNN inference, and 3) early-exit component. The input building component is responsible for preparing the input data required for the inference process. When entering a switch, if it is the first inference switch (① in Fig. 3), it builds the input data by concatenating field values extracted from the headers. Otherwise (② in Fig. 3), the input data for the layer can be obtained from the inference header. With the input data, the BNN inference component performs the inference procedure. The BNN model is a binarized model in which every weight is 0 or 1; thus, the model size and the corresponding memory consumption can be

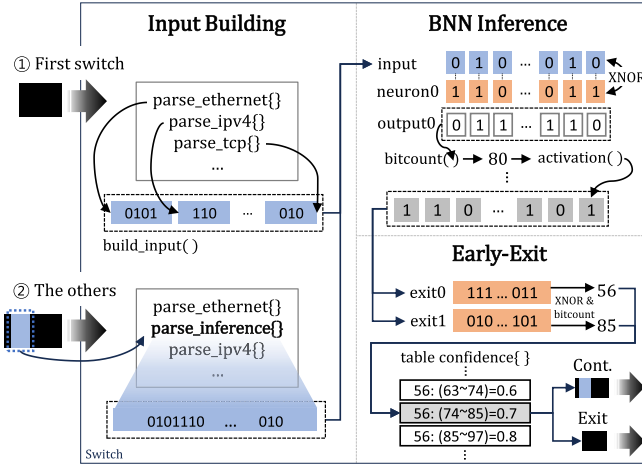


Fig. 3. Data plane architecture.

maximally reduced by compressing high-precision parameters to one bit. From the first neuron (neuron0 in Fig. 3), the bitwise XNOR operation is applied to the input data and the weight in the neuron. Next, the bitcount operation [26] is applied to count the number of 1s, and the value is activated to be 0 or 1. The process is repeated with every neuron, and intermediate data are generated by accumulating each result.

The early-exit component finally conducts the same bitwise XNOR and bitcount operations on the intermediate data and the weights of the early-exit neurons belonging to the early-exit branch. With the output values, the confidence score is obtained by looking up the confidence table. After that, if the confidence score is larger than a pre-defined threshold, the output is activated, and the result is marked on the packet header. In this case, the inference header becomes invalid. Otherwise, inference should continue, and thus the inference header is set to be valid to contain the intermediate data. Note that the packet becomes exited regardless of the value of the confidence score if it passes the last submodel.

Header and Forwarding Logic. For the tactical implementation, TINIEE employs a custom inference header and two custom Ethernet types, following the approach of [19]. The inference header contains two fields: `intermediate_data` and `inference_flag`. `intermediate_data` is a field that conveys the intermediate data of each layer. On the other hand, `inference_flag` is a field that identifies the number of submodels the packet has passed. When the packet passes each submodel, specifically, if it is not exited at the current node, `inference_flag` is incremented by 1. Note that no inference header is included before the packet arrives at the first submodel, so that the end devices (i.e., hosts) are transparent to the internal inference procedure. After being added to the first submodel, the inference header exists during the inference and is removed when exited. Meanwhile, we set the custom Ethernet types as `TYPE_INFER` (0×8845) and `TYPE_EXIT` (0×8890), to identify the current inference status, indicating whether the packet is in the middle of inference or has already exited. Note that `TYPE_INFER` and `TYPE_EXIT` are marked on the first inference device and the exiting inference device, respectively.

The detailed forwarding logic is as follows. If the Ether type is `TYPE_IPV4`, the packet is preferentially forwarded to the first submodel. Otherwise, if the Ether type is `TYPE_INFER`, meaning that the inference procedure is ongoing, the packet moves to the subsequent inference device. For the two cases, the inference routing table is applied, which regards `inference_flag` of the inference header as a match key and sets the forwarding port that can reach the $(\text{inference_flag} + 1)$ th submodel.¹ Lastly, if the Ether type is `TYPE_EXIT`, the packet is forwarded according to the normal routing table, which sets the destination IP address as a key value and determines the right port toward the destination.

Confidence Estimation. Estimating the confidence score in the data plane is a challenging task since it usually includes complex operations (e.g., exponentiation), which are far beyond the capability of the current PDP's arithmetic and logical unit (ALU) [22]. For example, one of the commonly used methods, Softmax, is calculated as

$$\Omega = \exp(y_{\max}) / \sum_{i \in L} \exp(y_i) \quad (1)$$

where L is the set of labels and $y_{\max}$ is the largest output value. To handle this, we use a table-based method where all possibilities are pre-calculated in a table. The rationale for this approach is that since the output value of BNN is discrete, the number of possible entries is feasibly small.

However, when including every possible combination of output values, the table size explodes as the number of labels grows, $\propto s^{|L|}$ where s is the size of the layer. This would lead to serious memory consumption and look-up time in the programmable network device. To mitigate this problem, we devise the following two strategies: 1) calibration-based confidence score approximation with dominant outputs and 2) granularity-based table compression.

First, it is required to reduce dimensions to fundamentally limit the table size. To this end, we devise an approximation method where only a few dominant output values are considered, and the remaining values are replaced with a single offset value. This design is supported by the characteristics of DNNs, where the output distribution for in-distribution inputs is typically highly concentrated on a few dominant classes, while the remaining classes collectively contribute only a negligible probability mass [49]. Note that we utilize the top-2 values that can capture the dominant portion of the confidence mass while incurring an acceptable number of combinations (i.e., s^2) for PDP tables. When using Softmax as the scoring metric, the confidence score cs becomes

$$\begin{aligned} cs(n) &= \frac{\exp(y_{(1)}^n)}{\sum \exp(y_i^n)} \\ &= \frac{\exp(y_{(1)}^n)}{\exp(y_{(1)}^n) + \exp(y_{(2)}^n) + |L|i = 3\exp(y_{(i)}^n)}, \end{aligned} \quad (2)$$

¹We, in fact, utilize internal metadata to store `inference_flag` and use the metadata as the actual key for inference routing. In this way, we can handle packets that go to the first submodel where the inference header does not exist. In this case, the value of the metadata is initialized to 0.

Algorithm 1 Exit Calibration Offset Searching (ECOS)**Input:**

Output data; $\mathbf{N} = \{(y_{(1)}^{(n)}, y_{(2)}^{(n)}, \dots, y_{(L)}^{(n)})\}_{n=1}^{|\mathbf{N}|}$
 Confidence threshold; c
 Step size; ΔR
 Stopping tolerance; τ

Output:

Offset minimizing exit mismatch at c ; R^*

```

1: Initialize  $R = 0$ ,  $R^* = 0$ , and  $best = +\infty$ 
2: Initialize  $noImp = 0$ 
3: while true do
4:   for  $n \in \mathbf{N}$  do
5:      $cs(n) = \exp(y_{(1)}^n) / \sum_{i \in L} \exp(y_{(i)}^n)$ 
6:      $ex(n) = \mathbb{1}[cs(n) \geq c]$ 
7:      $\hat{cs}(n) = \exp(y_{(1)}^n) / \{\exp(y_{(1)}^n) + \exp(y_{(2)}^n) + R\}$ 
8:      $\hat{ex}(n) = \mathbb{1}[\hat{cs}(n) \geq c]$ 
9:     Update  $m(R) = m(R) + \mathbb{1}[ex(n) \neq \hat{ex}(n)]$ 
10:    if  $m(R) < best$  then
11:      Set  $best = m(R)$ 
12:      Set  $R^* = R$ 
13:      Set  $noImp = 0$ 
14:    else
15:      Set  $noImp = noImp + 1$ 
16:    if  $noImp \geq \tau$  then
17:      break
18:     $R = R + \Delta R$ 
19: return  $R^*$ 

```

where $y_{(k)}^n$ denotes the k 'th largest output value of data n . Here, by replacing $\sum_{i=3}^{|\mathbf{L}|} \exp(y_{(i)}^n)$ with an offset R , we get²

$$\hat{cs}(n) = \frac{\exp(y_{(1)}^n)}{\exp(y_{(1)}^n) + \exp(y_{(2)}^n) + R}, \quad (3)$$

where $\hat{cs}(l)$ is the approximate confidence score of label l .

Therefore, our purpose is to find the offset which sufficiently compensates the approximated confidence score, i.e., $cs(l) \approx \hat{cs}(l)$. Specifically, since the ultimate goal of evaluating the confidence score is to decide on the early-exit, R should be the value that makes the exit tendency close to the original (i.e., exit calibration). To this end, we devise the ECOS algorithm to search for the proper exit calibration offset R , which is described in **Algorithm 1**. Using output data from the trained dataset, ECOS continuously searches for R until it no longer improves the calibration any further.

First, the algorithm parameters, current R , latest best offset R^* , and latest best calibration $best$ are initialized, respectively (line 1). Also, a parameter tracking whether calibration is continuously improved to judge whether to stop searching, $noImp$, is initialized (line 2). Using the output data $\mathbf{N}$ (line 4), the original confidence score and the corresponding exit decision are obtained (lines 5-6). In addition, the estimated confidence score with the current R is used to determine the

corresponding exit decision (lines 7-8). Using these values, the mismatch level of the current R , $M(R)$, is accumulated if the original and estimated exit decisions differ (line 9). After aggregating $M(R)$, it is compared with the latest best (i.e., minimum) mismatch value. If $M(R)$ is lower than the best value, $best$ and R^* are updated with the current R (lines 10-12). Furthermore, since an improved R is found, $noImp$ is reset to 0. On the other hand, if the current R is not better than R^* , $noImp$ is increased, and the searching progress is completed if further improvement does not occur for τ steps (lines 15-17). Lastly, if the iteration continues, R is increased by the searching step ΔR for the next iteration (line 18).

Algorithm 2 Generation of Rules for Adaptive Inference With Confidence Estimation (GRACE)**Input:**

Size of the last layer; s_l
 Granularity; g_{cs}
 Exit calibration offset; R

Output:

Rules for confidence score estimation

```

1: Initialize  $\mathbf{K}$  as a set of increasing integers  $[1, \dots, s_l]$ 
2: for  $s \in \mathbf{K}$  do
3:   Initialize  $c\_level = 0.5$ 
4:   Initialize  $start\_v = 0$  and  $finish\_v = 0$ 
5:   for  $s' \in \mathbf{K}$  and  $s' \geq s$  do
6:     Obtain confidence score  $cs$  with  $s, s', R$  from Eq. 3
7:     if  $cs \geq c\_level$  then
8:       if  $start\_v == 0$  then
9:         Set  $start\_v = s'$ 
10:      if  $cs > c\_level + g_{cs}$  then
11:        if  $finish\_v == 0$  then
12:          Set  $finish\_v = s' - 1$ 
13:        Generate  $(s, start\_v..finish\_v, c\_level)$ 
14:        Initialize  $start\_v = 0$  and  $finish\_v = 0$ 
15:        while  $cs > c\_level + g_{cs}$  do
16:          Update  $c\_level = c\_level + g_{cs}$ 

```

According to our test with the ISCXVPN 2016 [37] dataset, the exit mismatches of R^* are 2.1%, 3.4%, 3.1%, and 4.5% for classification tasks with 4, 5, 6, and 7 labels, respectively. This indicates that exit calibration offsets obtained by ECOS allow sufficiently accurate confidence score approximation while maintaining a reasonable table size (for 126 neurons and 15K entries) due to considering only two output values.

Second, to further compress the size of the table, we propose the GRACE algorithm to compactly generate rules for computing the confidence score. GRACE sets the granularity of the confidence score g_{cs} and expresses the entries using exact and range matching operations, as described in **Algorithm 2**. The key idea is to fix smaller output values and find the range of larger output values that make the level of the confidence score equal. For instance, if the granularity is 0.1, then four separate entries for 67 with 81, 82, 83, and 84, of which confidence scores are 0.802, 0.817, 0.832, and 0.845, respectively, can be merged into a single entry stating that 67 and 81 ~ 84 share an approximate confidence level of 0.8. In the P4 rule, we

²Note that R can be simply treated as 0 for binary classification.

express in this way: (67,81...84): Confidence (0.8). Using this approach, numerous combinations of output values can be compactly expressed as fewer ranges, and the degree of compactness can be adjusted by means of the granularity of the confidence score value (i.e., g_{cs}).

The detailed description of GRACE is as follows. We first initialize $\mathbf{K}$ as an increasing order of integers from 1 to s_l , which is the set of every possible output value (line 1). Then the rule generation loop is applied for the values in $\mathbf{K}$ (line 2). c_level is used to store the current confidence score level, which is initialized as the lowest possible value, 0.5 (line 3). $start_v$ and $finish_v$, initialized as 0, are edge values in the range of s' where the confidence score obtained from every s' and the current s is at the level of c_level (line 4). Then the inside loop is applied to get $start_v$ and $finish_v$ from c_level (line 5). Specifically, for the confidence score cs obtained with s and s' , it is checked whether it is the first case that cs satisfies c_level , which means that s' is $start_v$ (lines 6-10). After that, if cs crosses the current c_level while searching for larger values of s' , the previous value of s' (i.e., $s' - 1$) becomes $finish_v$. Consequently, the rule ($s, start_v..finish_v, c_level$) is generated, which means that if one of the output values is s and the other is between $start_v$ and $finish_v$, the confidence score is at the level of c_level (lines 12-15). Lastly, $start_v$ and $finish_v$ are initialized, and c_level is updated to the higher level where cs can be compared (lines 16-19).

Note that considering the layer with 126 neurons, the number of rules for the confidence estimation is 7875, 3368, 1015, 454, and 327 when g_{cs} is 0, 0.01, 0.05, 0.1, and 0.15, respectively. It indicates that GRACE can successfully compress the size of the table and control the degree of compression using g_{cs} .

B. Early-Exit-Aware Submodel Placement

Since we consider a situation where multiple inference devices exist, it is necessary to find a proper placement of submodels to prevent a significant amount of traffic caused by the exclusive forwarding chain between submodels. In addition, the property of our adaptive mechanism makes the submodel placement problem even more complicated since each flow has diverse inference results and the corresponding exit point. Taking these considerations into account, we formulate a submodel placement problem considering early-exit to minimize network traffic costs. To obtain a practical solution, we devise a low-complexity heuristic algorithm. Important notations are summarized in Table I.

Problem Formulation. We first define a binary decision variable $x_{n,m}$ that can be expressed as

$$x_{n,m} = \begin{cases} 1 & \text{if submodel } m \in \mathbf{M} \text{ is deployed on} \\ & \text{programmable network device } n \in \mathbf{N}; \\ 0 & \text{otherwise,} \end{cases} \quad (4)$$

where $\mathbf{M}$ and $\mathbf{N}$ denote the sets of submodels and programmable network devices, respectively. Note that we assume $|\mathbf{M}| \leq |\mathbf{N}|$, i.e., the number of programmable network devices is greater than or equal to the number of submodels.

TABLE I
SUMMARY OF NOTATIONS

Notation	Description
$\mathbf{M}$	Set of submodels
$\mathbf{F}$	Set of flows in the network
$\mathbf{P}_f$	Set of packets in the flow $f \in \mathbf{F}$
$\mathbf{N}$	Set of programmable network devices
$x_{n,m}$	Decision variable whether programmable network device $n \in \mathbf{N}$ is occupied by submodel $m \in \mathbf{M}$ or not
$\mathbb{I}_{p,m}^c$	Indication value whether packet $p \in \mathbf{P}_f$ continues inference on submodel $m \in \mathbf{M}$ when confidence threshold is c
$y_{p,m}^c$	Auxiliary variable whether packet $p \in \mathbf{P}_f$ exits on submodel $m \in \mathbf{M}$ when confidence threshold is c
m_p^c	Submodel where packet $p \in \mathbf{P}_f$ exits when confidence threshold is c
c	Confidence threshold for early-exiting
$cs_{p,m}$	Confidence score of packet $p \in \mathbf{P}_f$ on submodel $m \in \mathbf{M}$
g_{cs}	Granularity to express confidence score
s_m	Size of last layer of submodel $m \in \mathbf{M}$
s_p	Size of packet $p \in \mathbf{P}_f$
S_p	Source node of packet $p \in \mathbf{P}_f$
D_p	Destination node of packet $p \in \mathbf{P}_f$
$d_{x,y}$	Distance between two nodes x and y
T_o	Total traffic generated outside the forwarding chain between inference devices
T_i	Total traffic generated inside the forwarding chain between inference devices
T_n	Estimated total traffic when programmable network device n is selected for the exit point of considering flows
P_m	Selected programmable network device for submodel m
e_f	Indicator whether radical (1) or conservative (0) case

Since a submodel should be deployed on one of the programmable network devices, we have a constraint

$$\sum_{n \in \mathbf{N}} x_{n,m} = 1, \quad \forall m \in \mathbf{M}. \quad (5)$$

Furthermore, every programmable network device can hold up to a single submodel, which can be expressed as

$$\sum_{m \in \mathbf{M}} x_{n,m} \leq 1, \quad \forall n \in \mathbf{N}. \quad (6)$$

During the process, a submodel can be utilized only if the packet has not exited the previous submodel. Therefore, $\mathbb{I}_{p,m}^c$ denoting whether packet $p \in \mathbf{P}_f$ continues the inference on submodel $m \in \mathbf{M}$ when the confidence threshold is c and can be expressed

$$\mathbb{I}_{p,m}^c = \begin{cases} 1 & \text{if } cs_{p,m-1} < c \text{ or } m = 1; \\ 0 & \text{otherwise,} \end{cases} \quad (7)$$

where $cs_{p,m}$ is the confidence score of packet $p \in \mathbf{P}_f$ on submodel $m \in \mathbf{M}$. Note that the first submodel is unconditionally used for the start of the inference procedure.

The exit point of the packet corresponds to the first point where $\mathbb{I}_{p,m'}^c = 0$ and $\mathbb{I}_{p,m}^c = 1$ for every $m < m'$. Therefore, m_p^c denoting the index of the submodel where packet $p \in \mathbf{P}_f$ exits (i.e., exit point index) and $y_{p,m}^c$ denoting whether packet $p \in \mathbf{P}_f$ exits the submodel $m \in \mathbf{M}$ when the confidence threshold is c , which can be expressed as

$$m_p^c = \sum_{k=1}^{|\mathbf{M}|} \prod_{m=1}^k \mathbb{I}_{p,m}^c, \quad \forall p \in \mathbf{P}_f, \quad (8)$$

and

$$y_{p,m}^c = \begin{cases} 1 & \text{if } m = m_p^c; \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

Now we formulate the network traffic generated by flows that traverse different sets of submodels and have different source–destination pairs. Since each flow preferentially passes submodels in order until it meets its exit point, its total network traffic can be decomposed into two components: 1) the traffic outside the submodel chain, including the paths from the source to the first submodel and from the exiting submodel to the destination, and 2) the traffic inside the submodel chain, corresponding to the forwarding traffic from the first submodel to the exiting submodel.

Let T_o be the network traffic generated outside the chain of inference devices by every packet. Then it can be characterized by the traffic generated on the following two subpaths: 1) from the source to the first inference device and 2) from the inference device where the packet exits (i.e., the exit point) to the destination. Therefore, we have

$$T_o = \sum_{f \in \mathbf{F}} \sum_{p \in \mathbf{P}_f} \sum_{n \in \mathbf{N}} s_p x_{n,1} d_{S_p,n} + \sum_{f \in \mathbf{F}} \sum_{p \in \mathbf{P}_f} \sum_{m \in \mathbf{M}} \sum_{n \in \mathbf{N}} s_p x_{n,m} y_{p,m}^c d_{n,D_p}, \quad (10)$$

where s_p denotes the size of packet $p \in \mathbf{P}_f$, and $d_{x,y}$ indicates the distance between node x and node y .

Moreover, T_i denotes the network traffic generated inside the forwarding chain between inference devices by each packet, which can be expressed

$$T_i = \sum_{f \in \mathbf{F}} \sum_{p \in \mathbf{P}_f} \sum_{m \in \mathbf{M}} \sum_{n \in \mathbf{N}} \sum_{n' \in \mathbf{N}} (s_p + s_m) \mathbb{1}_{p,m+1}^c x_{n,m} x_{n',m+1} d_{n,n'}, \quad (11)$$

where s_m indicates the size of the last layer of submodel $m \in \mathbf{M}$ (i.e., the size of the intermediate data).

Since our goal is to minimize the total network traffic, the corresponding optimization problem can be formulated as

$$\mathbf{P1} : \text{minimize } T_o + T_i \quad (12a)$$

$$\text{subject to (2) – (9).} \quad (12b)$$

Theorem 1: The formulated problem **P1** is NP-hard.

Proof: We prove NP-hardness by showing the reduction of the Shortest Hamiltonian Path problem to the restricted **P1**. We set the sub-problem of **P1** as a decision version of the submodel placement problem, which decides whether there exists a placement of submodels on a given sub-network, with the number of programmable network devices equal to that of submodels, such that the traffic cost inside the forwarding chain is at most a given threshold. In this sub-problem, the traffic outside the chain is neglected, early exit is disabled, and the inter-submodel traffic is set to a constant s .

The Shortest Hamiltonian Path problem can be defined as follows: given a complete graph $G = (V, E)$ with a nonnegative edge cost $c(u, v)$ for each pair $(u, v) \in V \times V$ and a bound B , decide whether there exists a Hamiltonian path that visits every vertex exactly once with a total cost of at most B .

Now we reduce the Shortest Hamiltonian Path problem to the restricted **P1**. First, we perform a one-to-one mapping from

each vertex $v \in V$ to a programmable network device $n \in \mathbf{N}$. Here, we also set the number of submodels to that of vertices, i.e., $|\mathbf{M}| = |V|$. Furthermore, we map the edge cost to the traffic generated between two devices in the network as $c(u, v) = s \cdot d_{n(u),n(v)}$, $\forall u, v \in V$, which corresponds to the multiplication of constant inter-node traffic s by the distance.

For any Hamiltonian path $(v_1, \dots, v_{|V|})$, we can construct a feasible solution by assigning submodel m to programmable network device $n(v_m)$, yielding an objective value $\sum_{m=1}^{|V|-1} c(v_m, v_{m+1})$. Thus, there exists a Hamiltonian path with cost at most B if and only if there exists a feasible placement whose objective value is at most B in the restricted **P1**. Since the transformation is polynomial-time and the Shortest Hamiltonian Path problem is NP-complete [33], we conclude that the restricted **P1**, and consequently **P1** is NP-hard. ■

Algorithm 3 Early-Exit-Aware Submodel Placement (ESP)

Input:

Set of submodels; $\mathbf{M}$

Set of flow request messages; $\mathbf{F}_r$

Set of programmable network devices $\mathbf{N}$

Output:

Submodel placement solution; $\{P_1, \dots, P_{|\mathbf{M}|}\}$

- 1: Initialize T_n as 0 for $\forall n \in \mathbf{N}$
- 2: Initialize P_m as 0 for $\forall m \in \mathbf{M}$
- 3: Obtain S_f , D_f , s_f , and $\tilde{m}_f^c$ from $\forall f \in \mathbf{F}_r$
- 4: **for** $m \in \mathbf{M}$ **do**
- 5: **for** $f \in \mathbf{F}_r$ **do**
- 6: **if** $(m == \tilde{m}_f^c \wedge e_f == 1) \vee (m \leq \tilde{m}_f^c \wedge e_f == 0)$ **then**
- 7: **for** $n \in \mathbf{N}$ **do**
- 8: Temporarily assume $P_m = n$
- 9: **if** $m = 1$ **then** ▷ Exit at the front
- 10: $T_o = s_f d_{S_f, P_1} + s_f d_{n, D_f}$
- 11: $T_i = 0$
- 12: **else** ▷ Pass submodels and exit at m
- 13: $T_o = s_f d_{S_f, P_1} + s_f d_{n, D_f}$
- 14: $T_i = \sum_{m'=1}^{m-1} (s_{m'} + s_f) d_{P_{m'}, P_{m'+1}}$
- 15: Set $T_n = T_o + T_i$
- 16: Set $P_m = \operatorname{argmin}_{n \in \mathbf{N}} T_n$ ▷ Select the best device
- 17: Set $T_{P_m} = \infty$ ▷ Prevent duplicated device selection
- 18: Initialize T_n as 0 if $T_n < \infty$ for $\forall n \in \mathbf{N}$
- 19: Iterate above for $e_f == 1, 0$ and select solution with lesser

$$\sum_{f=1}^{|\mathbf{F}_r|} (s_f d_{S_f, P_1} + s_f d_{P_{|\mathbf{M}|}, D_f} + \sum_{m=1}^{|\mathbf{M}|-1} (s_m + s_f) d_{P_m, P_{m+1}})$$

ESP Algorithm. Since the formulated problem is NP-hard, it is necessary to find a practical algorithm to obtain a solution that can be run in a feasible time. To this end, we devise a heuristic algorithm called ESP, described in **Algorithm 3**. We use a greedy approach to sequentially select the location of each submodel that generates the least network traffic by assuming that the selected location is the last submodel (i.e., exit point) for the considering flows. Here, we consider the following two cases regarding the scope of considering flows:

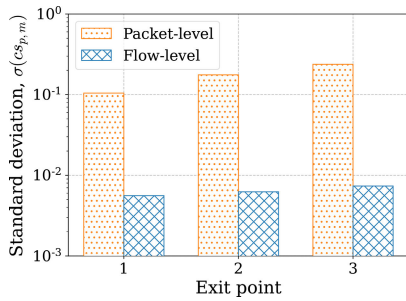


Fig. 4. Standard deviation of confidence score for packet-level (Packets) and flow-level (Flows) on three different exit points.

1) only flows that terminate inference (i.e., early-exit) on the submodel and 2) all flows that continue inference on the submodel, including those that may early-exit. The first can be regarded as a radical case to reflect early-exiting, as it can reinforce the early-exit effect by setting the position advantageous to exiting flows; however, it can cause even more traffic by making the location biased towards a few flows. To prevent the aforementioned side effect, we compare it with a conservative case considering every flow passing the submodel and choose a better solution.

At first, we initialize two variables T_n and P_m to 0. T_n is a variable that stores the estimated network traffic when the programmable network device n is selected (line 1). P_m represents the solution (i.e., location) for the submodel m (line 2). Then we extract some useful information from the collected flow requests, which are composed of the source S_f , destination D_f , flow size s_f , and the exit point $\tilde{m}_f^c$ (line 3). Note that $\tilde{m}_f^c$ is not included in the flow request message, unlike the others; it is obtained by conducting an inference using the flow request message with the trained model. Since packets in the identical flow share similar input features [41], the deviations of the confidence score and exit tendency are sufficiently low, giving complete representativeness to the flow request message. The detailed analysis of this is presented in Fig. 4, showing that the standard deviation of the confidence score is much lower when calculated at the flow level than at the packet level.

After the preparation, the submodel placement procedure is executed following the order of submodels (line 4). For every flow, it is first checked whether it is the exiting flow in the radical case (denoted as $e_f = 1$) or the passing flow in the conservative case (denoted as $e_f = 0$) (i.e., one of the flows to consider). For the radical case, the considering flows only include flows for which the exit point is exactly the current submodel (i.e., $m == \tilde{m}_f^c$). On the other hand, the considering flows in the conservative case cover flows that exit behind the submodel (i.e., $m \leq \tilde{m}_f^c$). For the considering flows of each case, the network traffic is estimated for every programmable network device (lines 5-7). Here, although the placement for the current submodel m (i.e., P_m) is not decided yet, P_m is temporarily assumed to be n to easily calculate expected total traffic when P_m is actually decided as device n (line 8).

To estimate the total traffic, we assume that all flows exit the current submodel to focus on the considered flows. With this assumption, T_o and T_i are calculated and gathered in T_n . Note that the programmable network devices selected in the previous submodels are used to estimate the current T_n

(lines 9-15). After obtaining T_n for every programmable network device and for every requested flow, the node with the lowest value of T_n is selected as the inference device for the submodel and stored in P_m (line 16). To prevent the chosen node from being redundantly selected, we permanently set T_n of the selected node to infinity and initialize the others to 0 (lines 17-18). Finally, as mentioned, the above process from line 1 to line 16 is iterated for both radical and conservative cases, and the solution with the lesser estimated traffic is selected (line 19).

Complexity Analysis. There are three inclusive loops that iterate $|\mathbf{M}|$, $|\mathbf{F}|$, and $|\mathbf{N}|$ times, respectively. To estimate the network traffic, it requires $|\mathbf{M}|$ times of distance³ calculation with $\mathcal{O}(|\mathbf{N}|^2)$. In addition, we iterate every step twice, as in the radical and conservative cases. Therefore, the time complexity of **Algorithm 3** is $\mathcal{O}(2 \cdot |\mathbf{M}|^2 \cdot |\mathbf{F}| \cdot |\mathbf{N}|^3)$, which is a form of a polynomial function for the given input and much lower than $\mathcal{O}(2^{|\mathbf{M}| \cdot |\mathbf{N}|})$, which is the complexity of solving problem **P1** using the brute-force method. Since the complexity is sufficiently low, it can be adopted in the real world where flow traffic and requests change dynamically.

V. PERFORMANCE EVALUATION

We implemented⁴ TINIEE in both BMv2 [36] software switch and the Intel Tofino [27] hardware switch. We trained the early-exit model in Pytorch [40], consisting of a convolutional layer as the input layer and three fully-connected layers with three classifying layers (two of which are early-exit layers). The model is divided into three submodels m_1 , m_2 , and m_3 . Each submodel includes a fully-connected layer and a classifying layer, with the input layer exceptionally included in the first submodel. By default, we use the ISCXVPN 2016 [37] dataset, which contains real-world VPN and non-VPN traffic, and we strictly split it for training and testing data. In the dataset, flows are identified with 5-tuples, and it is assumed that the first packet of each flow is used as the flow request message. Input data for training and inference is composed of the IP protocol ID, IP source and destination addresses, IP header length, L4 ports, and TCP control flags, the total length of which is 126 bits. For the calculation of the confidence score, we use the Softmax function and set g_{cs} to 0.01. To simulate network-wide orchestration performance, we composed three network topologies from the real-world in the Mininet [35] environment composed of BMv2 switches: 1) Italian topology with 10 nodes [45], 2) NSFNET topology with 14 nodes [46], and 3) Japanese topology with 14 nodes [47] where every node is a programmable switch. In the experiment, every host sending or receiving packet flows is randomly distributed and attached to one of the BMv2 switches. Both inference and normal routing tables are generated using the Dijkstra [34] algorithm and are inserted into every switch using P4runtime. Additionally, Tofino implementation employs the exactly same number of multiple recirculations (i.e., forcing a packet to re-enter the pipeline iteratively) with neurons due to strictly restricted pipeline resources (e.g., stage, memory), following current works on Tofino-based DNN implementation [13], [31].

³The complexity of distance calculation using the classical Dijkstra algorithm is known to be $\mathcal{O}(|\mathbf{V}|^2)$, where $\mathbf{V}$ is a set of vertices [34].

⁴Both versions are available at <https://github.com/keemeew/TINIEE>

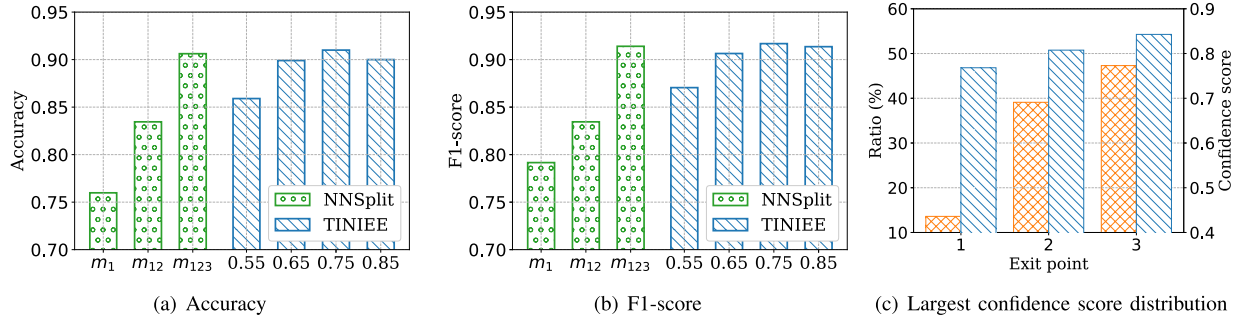


Fig. 5. In-network classification results. (a) Accuracy performance, (b) F1-score performance, (c) Ratio of the largest confidence score (left bar) and the corresponding average confidence score (right bar) of exit points.

Identical BNN inference and early-exit functionalities are realized in the Tofino version.

To check the effectiveness of TINIEE, our evaluation focuses on the following two aspects: 1) in-network classification performance and 2) network performance. In-network classification performance includes classification metrics (i.e., accuracy and F1-score) and the exit tendency affected by various factors (i.e., c and g_{cs}). Meanwhile, for network performance, we measure the network cost in terms of network traffic, end-to-end packet latency, load balancing (i.e., throughput imbalance and maximum link utilization), and flow completion time (FCT). These two aspects show that TINIEE can provide competitively accurate classification while minimizing network cost during the in-network inference process.

A. In-Network Classification Performance

To evaluate the classification performance, we compare the proposed adaptive in-network inference scheme with NNsplit [19], which is an in-network inference framework that dispersively splits and deploys multiple submodels on programmable network devices. Since it only supports static inference, every packet should pass through each submodel sequentially and be classified by the last submodel in the scheme.

Classification Performance. Fig. 5 shows the classification performance (i.e., accuracy and F1-score) for the static inference case (i.e., NNsplit) with different model compositions and for the adaptive inference case (i.e., TINIEE) with varying confidence thresholds. Here, m_{xy} denotes the model that is the conjunction of submodels m_x and m_y , and the intermediate early-exit layer is ignored. For example, m_{123} is the joint model of submodels m_1 , m_2 , and m_3 . In Fig. 5, it can be seen that both the accuracy and the F1 score increase in the order of m_1 , m_{12} , and m_{123} for the case of NNsplit. This is because, as more submodels are combined, the number of model parameters that can extract the data features becomes larger, leading to an improvement in classification performance. On the other hand, for the case of TINIEE, the accuracy and the F1-score become higher as c increases to some extent (i.e., $c = 0.75$). The reason for the increasing performance trend is that a larger c induces more confident packets to exit. However, interestingly, classification performance seems to degrade in $c = 0.85$. This is because, as shown in Fig. 5(c), the most backside exit point (i.e., 3) is not the only point where a packet can achieve the most confident result. Specifically, more than 50% of the packets have the highest confidence score at exit

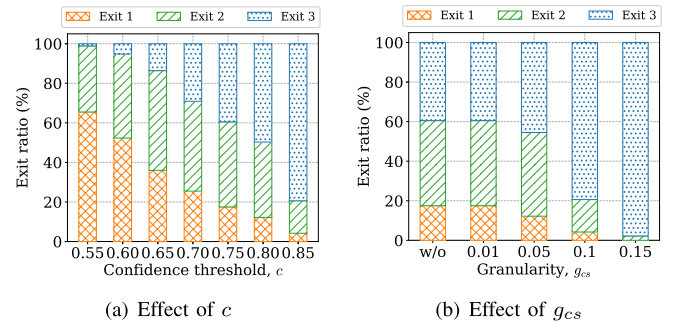


Fig. 6. Exit ratio of each exit point.

points 1 and 2, and the average confidence score of those cases is approximately 0.8. Therefore, if we set c too large, the packets cannot exit at the point where they can obtain the most confident result, leading to performance degradation.

Exit Tendency according to c and g_{cs} . Fig. 6(a) shows the effect of c on the exit ratio, demonstrating that the exit ratio tends to be higher for the backside exit point as c increases. This is because, as c increases, it becomes harder to leave the fore-exit points. Moreover, considering Fig. 5 and Fig. 6(a) simultaneously, TINIEE can achieve high classification performance relative to the exit tendency. For example, when $c = 0.65$, both the accuracy and the F1 score of TINIEE are comparable to NNsplit where every submodel is combined (i.e., m_{123}), although more than 80% of the packets can exit before the last exit point. This is because TINIEE carefully decides which packet to exit based on the confidence score, making the expected fidelity of classified results guaranteed.

Meanwhile, Fig. 6(b) presents the effect of g_{cs} on the exit ratio when $c = 0.75$, which shows that fewer packets exit as g_{cs} becomes larger. This is because the GRACE algorithm binds c_level from the bottom (i.e., 0.5), which has the effect of rounding the actual confidence score, thus making the confidence threshold relatively larger in proportion to g_{cs} . Note that for $g_{cs} = 0.01, 0.05, 0.1, 0.15$, the corresponding F1-scores are 0.9168, 0.9203, 0.9149, 0.9141, and the accuracies are 0.910, 0.913, 0.907, 0.906, respectively, with the overall performance fluctuation remaining within approximately 1%.

B. Network Performance

To compare the orchestration performance, we evaluate ESP in comparison with DINC [43], as it is a well-known

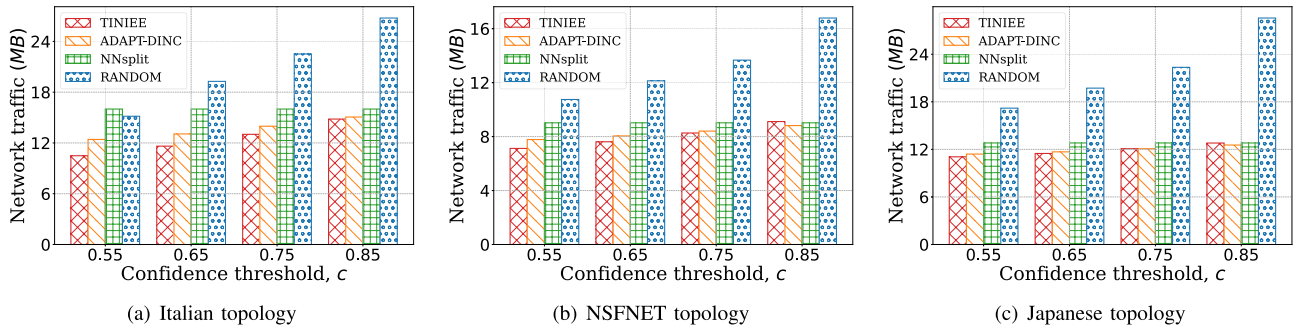


Fig. 7. Network traffic of difference cases according to confidence threshold on three topologies.

distributed algorithm that divides a PDP program and finds proper locations according to the predefined objective. To fit our problem, we set the objective of minimizing traffic costs while considering the forwarding chain of the entire submodels. For a fair comparison, we used a brute-force method to obtain the solution of DINC. Note that DINC itself does not include any in-network inference mechanism; however, we adopt both adaptive and static inference methods to clearly demonstrate the effectiveness of ESP. Similarly, we adopt DINC to NNsplit, which itself has no orchestration mechanism, to shed light on the efficacy of the proposed adaptive in-network inference. Consequently, we have four comparison schemes: 1) TINIEE where adaptive in-network inference is conducted and submodels are deployed by ESP, 2) ADAPT-DINC where adaptive in-network inference is applied and submodel placement is conducted by DINC, 3) NNsplit where static in-network inference is executed and submodels are deployed by DINC and 4) RANDOM where adaptive in-network inference is conducted and submodels are located in arbitrary devices.

Network Traffic. The network traffic is shown in Fig. 7. At first, when comparing ADAPT-DINC with NNsplit, ADAPT-DINC shows lower network traffic than NNsplit for all c values and in all topologies. This is because adaptive in-network inference allows packets to pass through fewer submodels, leading them to traverse a shorter distance than the static case with identical placement. Furthermore, TINIEE reduces more network traffic than ADAPT-DINC, achieving a reduction of up to 34.48% in network traffic compared to NNsplit. In particular, TINIEE is shown to be superior to the rest of the schemes when $c \leq 0.75$ in all topologies. This indicates that the ESP algorithm, reinforcing early-exiting, is beneficial in reducing overall network traffic in most cases. However, when $c = 0.85$, TINIEE has no advantage compared to ADAPT-DINC and NNsplit and is even slightly worse in Fig. 7(b) and 7(c). This is because only 20% of the packets exit before the end of the submodel when $c = 0.85$, which is too small to benefit from the improved early-exit effect, and the negative effect of biasing locations to a few flows becomes significant. Meanwhile, except for the case where $c = 0.55$ in the Italian topology, the performance of RANDOM is the worst, even though adaptive in-network inference is applied. This suggests that careful orchestration of the submodels is required to make adaptive inference in the network effective.

End-to-end Latency. The distribution of end-to-end delays of the packets is described in Fig. 8. At first, Fig. 8(a) shows

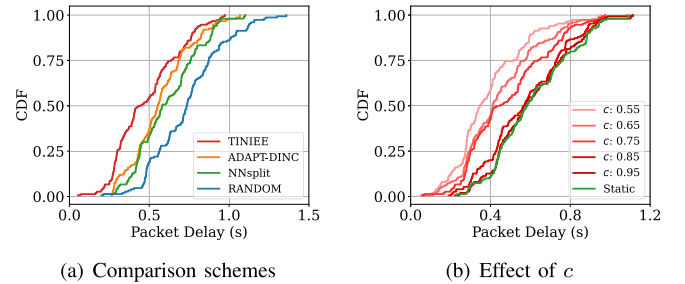


Fig. 8. Packet delay distributions on Italian topology.

the packet delay performance when $c = 0.75$ for every adaptive scheme. In Fig. 8(a), it can be seen that both TINIEE and ADAPT-DINC have shorter packet delays than NNsplit. This is because they both utilize early-exit, which allows packets to escape from the forwarding chain earlier, leading to a shortened end-to-end path length. Furthermore, TINIEE has more low-delay packets than ADAPT-DINC, with the average packet delay reduction ratios of TINIEE and DINC compared to NNsplit being 17.8% and 4.2%, respectively. This indicates that the strategy of TINIEE to place submodels closer to the destination for exited packets can effectively reduce the overall latency. Meanwhile, RANDOM has the worst packet delay distribution, implying that packets traverse long distances due to unsuitable locations of submodels, even though some packets benefit from being exited.

Moreover, Fig. 8(b) represents the effect of c when ESP is utilized as the placement algorithm. In Fig. 8(b), from $c = 0.55$ to $c = 1$ (i.e., static inference), the distribution has a strong tendency to increase the proportion of high-latency packets as c grows larger. This implies that the proposed adaptive in-network inference can precisely control the trade-off. Moreover, the large gap between the distribution curves of $c = 0.75$ and the higher thresholds suggests that setting the proper threshold (i.e., $c = 0.75$) can effectively reduce the network cost while preserving sufficient accuracy.

Throughput Imbalance. Fig. 9 shows the imbalance between the minimum and maximum throughputs of links that are attached to the first inference device in the Italian topology. This metric is useful for measuring the load balance level in the network [44], which can be calculated as $(\max(T_l) - \min(T_l)) / \text{AVG}(T_l)$, where T_l represents the throughput of link l , indicating the difference between the extremes relative to the average throughput. Note that we divided the sum of the packet

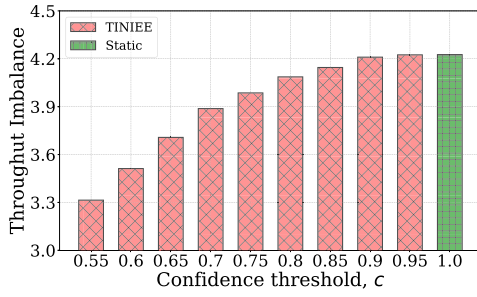


Fig. 9. Throughput imbalance of links on Italian topology.

lengths passing through each link during the experimental time to obtain the throughput. In Fig. 9, the throughput imbalance continuously decreases as the value of c becomes smaller. This indicates that the proposed adaptive inference mechanism can effectively distribute intensive traffic on the forwarding chain to the other links.

Maximum Link Utilization. Following the previous insight that TINIEE has a beneficial effect in mitigating traffic bottlenecks, we measure the maximum link utilization for each comparison scheme in every topology, as shown in Fig. 10. To evaluate this, we set the link capacity equal to the sum of every flow's data rate in the network. Firstly, the results of NNsplit are nearly 100% of maximum link utilization for all cases. This is because the forwarding chain between submodels becomes an obvious bottleneck, as every packet follows the inference route until it reaches the last submodel. On the other hand, ADAPT-DINC shows less maximum link utilization than NNsplit for every value of c . The reason is that the proposed adaptive in-network inference method distributes network traffic to multiple paths before it is concentrated on the forwarding chain. Furthermore, it can be seen that TINIEE can reduce maximum link utilization more than ADAPT-DINC when c is sufficiently small, which can reduce up to 52.7% of maximum link utilization compared to NNsplit in Fig. 10(c). This is because TINIEE places the submodels in the best locations of actually exiting packets, preventing them from passing through additional links. Meanwhile, for RANDOM, it can be shown that the maximum link utilization exceeds 100% in every case and on every topology. This result indicates that uncoordinated submodel placement causes packets to go around some of the nodes redundantly, leading to excessive link utilization and congestion.

Flow Completion Time. Lastly, Fig. 11 shows the FCT performance, where we average the completion time of every flow and normalize each with the result of NNsplit. In Fig. 11, it is reported that TINIEE has the lowest FCT, except for the case $c \geq 0.85$ in Fig. 11(b), with a maximum of 17.8% FCT reduction compared to NNsplit in the Italian topology. This result corresponds to the network traffic result in Fig. 7 and the maximum link utilization result in Fig. 10. Thus, it can be concluded that the effect of TINIEE in reducing the distance of the traversing path and the intensive load on the forwarding chain together leads to minimizing the average FCT. However, for $c \geq 0.9$, the performance of TINIEE is worse than that of ADAPT-DINC and even NNsplit. This is due to the greedy nature of the ESP algorithm, which gives priority to 1) the exiting and 2) the larger flows when deciding the placement

of submodels, resulting in disadvantages relative to the other flows, which are the majority when c is excessively large.

C. Performances Under Various Configurations

Performance Under Different Flow Request Messages.

As checked in Fig. 4, packets in the identical flow tend to have minor deviations in confidence scores, leading to similar exit points. According to our analysis, 85%, 81%, 85%, and 90% of flows have exactly the same exit points among packets, while 35%, 42%, 42%, and 31% of packets have different exit points compared to the first packet (i.e., flow request message) for the rest of the flows on average when $c = 0.55, 0.65, 0.75, 0.85$, respectively. This indicates that utilizing the flow request message to set the representative exit point is reasonable for the majority of flows; however, some flows might be confused when the flow request message does not represent the flow exit point.

To explicitly check the impact of early-exit deviance in identical flows, we further measure the difference in network traffic when the flow request message is randomly selected for each flow, as described in Fig. 12. In this test, we consider 100 random cases and examine the worst and average performance differences (denoted as TINIEE (Worst) and TINIEE (Average) in the figures) compared to the case when using the first packet as the flow request message (i.e., TINIEE).

Fig. 12 shows the network traffic differences compared to TINIEE. In most situations, it can be seen that the random selection of flow request messages causes only marginal changes of up to 1.9% difference for the worst case, still being better or at least comparable to ADAPT-DINC. Furthermore, the average performance of TINIEE with random flow request messages shows a negligible difference of less than 0.1% compared to TINIEE. One of the main reasons for the robust performance is the minor deviation of exit points of packets in each flow; moreover, the design of the ESP algorithm, which compares the performances between radical and conservative cases guarantees stable performance without being confused by some changes in the exit points of flows.

Meanwhile, interestingly, we also found that there exist cases where random performance surpasses the original TINIEE. Although marginal, average network traffic becomes reduced by 0.2%, 0.03%, and 0.05% for Italian, NSFNET, and Japanese topologies when $c = 0.85, 0.75, 0.65$, respectively. This is because the first packet does not have the majority exit point for a few flows, making the flow request message change beneficial. However, since cases where the flow request message exactly matches the majority or identical exit point are dominant, this impact is checked to be marginal.

Performance Under Different Numbers of Submodels.

We verify the effectiveness of TINIEE under various numbers of submodels M , as described in Fig. 13. Note that the composition of the cases with 2 and 3 submodels is identical to m_{12} and m_{123} , respectively, and the case with 4 submodels has a fully-connected layer added to m_{123} . Furthermore, the accuracies and F1-scores are 0.87, 0.90, 0.93, and 0.87, 0.92, 0.93 for the three cases, respectively, when $c = 0.65$.

Fig. 13 shows the network traffic performance of different cases according to varying numbers of included submodels when $c = 0.65$. On the whole, as the number of submodels increases, the network traffic becomes larger regardless of the

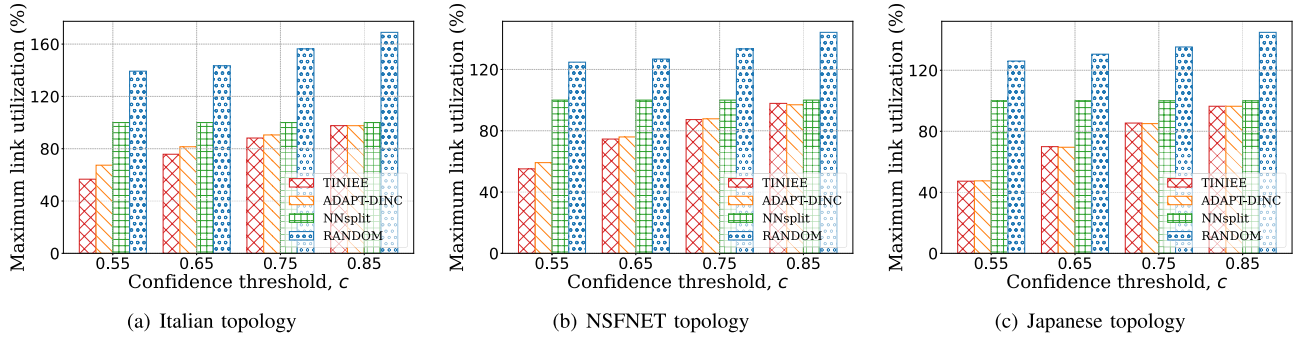


Fig. 10. Maximum link utilization of difference cases according to confidence threshold on three topologies.

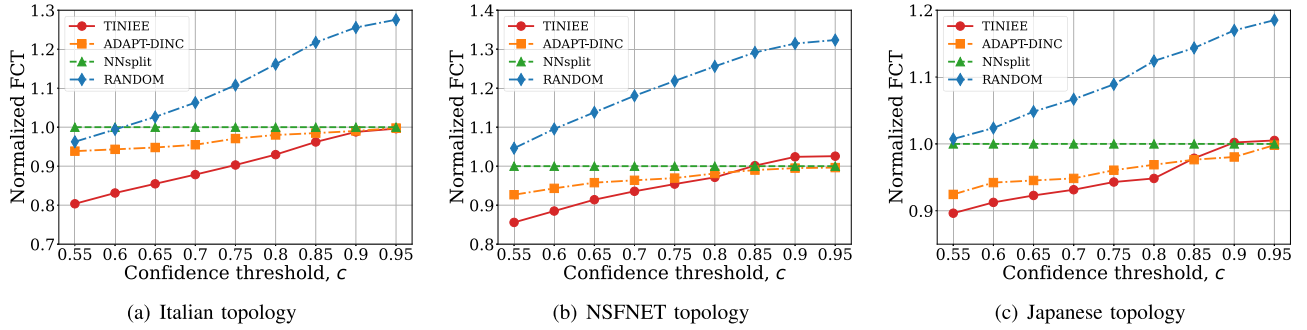


Fig. 11. FCT of difference cases according to confidence threshold on three topologies.

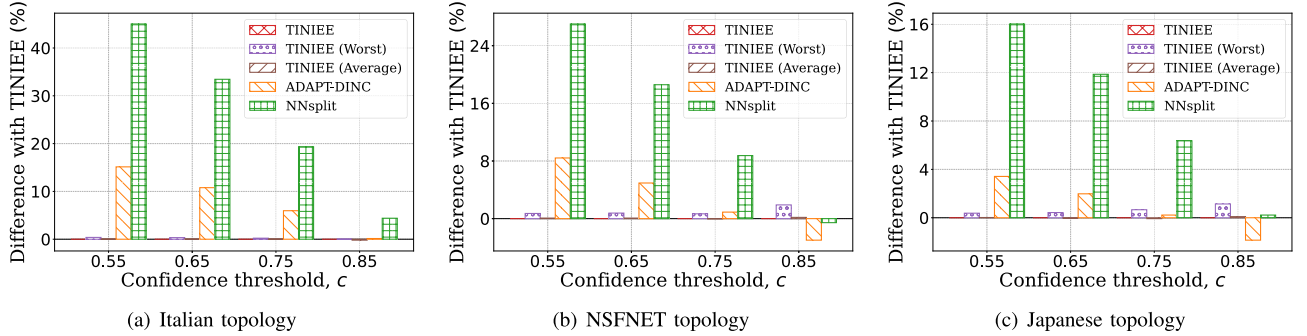


Fig. 12. Network traffic difference of cases including TINIEE with randomly selected flow request messages.

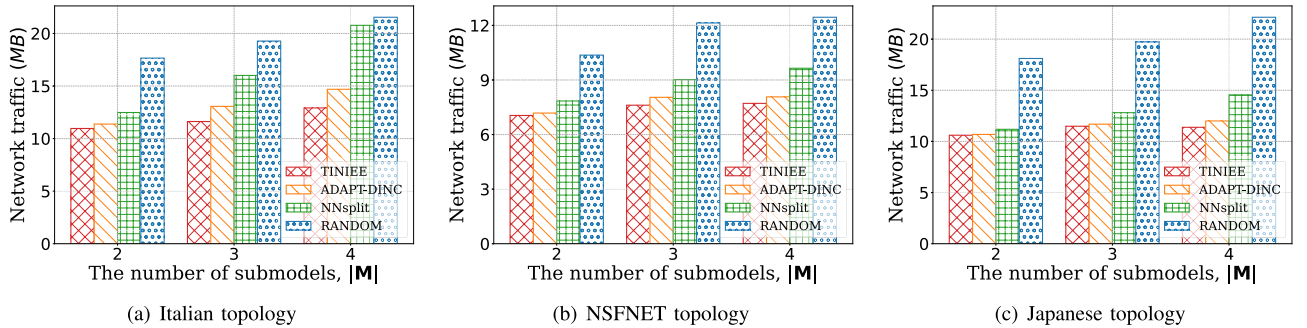


Fig. 13. Network traffic according to varying numbers of submodels when $c = 0.65$.

network topologies. This is due to the longer traversing paths when more submodels are included. Note that this can be seen as the trade-off of utilizing a larger model to secure better in-network classification performance. Furthermore, it is notable that the differences in network traffic between schemes

become more vivid as the number of submodels increases. For example, the network traffic reduction of TINIEE compared to Adapt-DINC and NNsplit in the Italian topology is 3.67%, 12.16%, and 12.08%, 37.82% when the numbers of submodels are 2 and 4, respectively. The main reason is that a longer

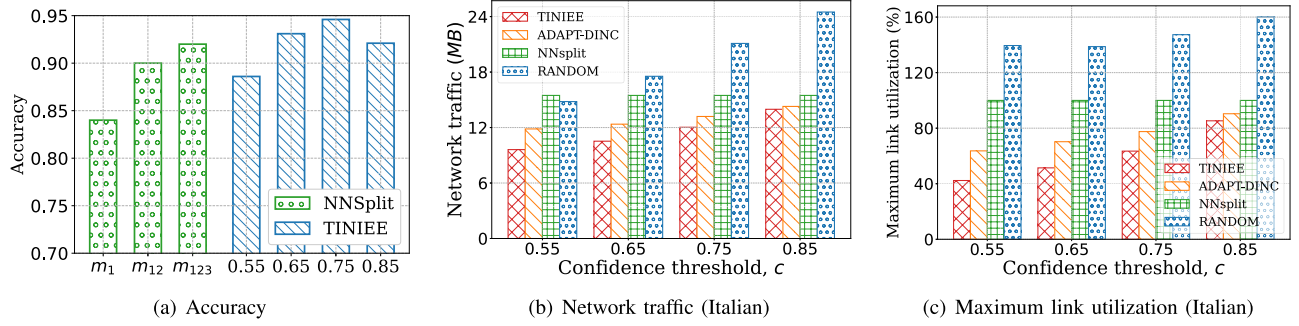


Fig. 14. Comprehensive performances when tested with ISCXTor 2016 dataset.

forwarding chain of submodels inevitably makes the impact of early-exiting even greater; in addition, wisely considering the exit tendency during submodel placement (i.e., the ESP algorithm) becomes more critical, which consistently causes a larger gap between TINIEE and Adapt-DINC.

Performance Under Different Task. To support the generality of the proposed scheme, we conduct the experiment with a newly added task, which is to classify Tor traffic [39]. Tor is similar to VPN in that it is a traffic encryption technique; however, unlike a VPN's single-hop tunneling, Tor employs multi-hop onion routing with fixed-size cells and traffic obfuscation mechanisms, resulting in substantially different packet size distributions and traffic patterns. For this test, we utilized the ISCX Tor/non-Tor [38] dataset, where the non-Tor traffic is shared with benign traffic from the VPN dataset, and Tor traffic is collected from five real-world users. Note that the rest of the configurations, such as training-test data split, model composition, input features, and granularity, are exactly identical to our default settings.

First, the classification performance is shown in Fig. 14(a). The overall trend is similar to the VPN case for both NNSplit and TINIEE; however, it is remarkable that TINIEE shows explicitly better accuracy (i.e., 2.5%) when $c = 0.75$ than NNSplit with m_{123} . Furthermore, as the confidence threshold becomes excessively high, the classification performance clearly decreases. This can be interpreted as the optimal exit point of each packet is not always the most rearward submodel, and thereby forcing it to be exited on the later submodel would have a negative impact on performance. In fact, a similar result has already been reported in the VPN case (see subsection A). Overall, it can be said that TINIEE in the Tor traffic classification demonstrates more pronounced competitiveness in terms of accuracy.

Moreover, Fig. 14(b) and (c) describes the network traffic and maximum link utilization on Italian topology, respectively. Interestingly, in both cases, not only the relationship of superiority remains the same, but also the differences between comparison schemes are larger than the VPN cases. For example, when $c = 0.65$, the network traffic and the maximum link utilization reduction of TINIEE compared to Adapt-DINC and NNSplit are 32.00% and 14.90%, respectively, which is higher than 11.02% and 7.03% of the previous cases. The reason can be found in Fig. 15, where the ratio of packets that exit in fore submodels is even larger than we checked in Fig. 6(a). Consequently, these results indicate that TINIEE shows coherent performances for the different task and dataset.

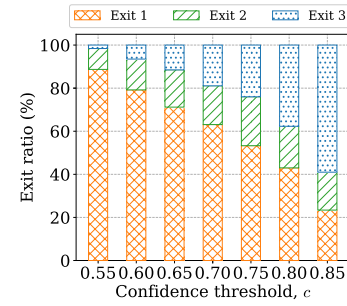


Fig. 15. Exit ratio tested with ISCXTor 2016 dataset.

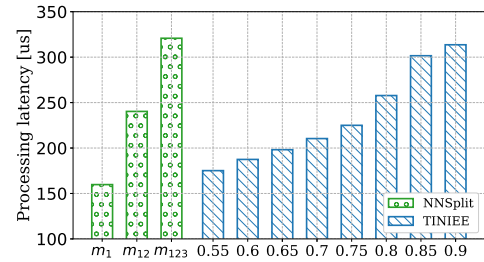


Fig. 16. Processing latency on Intel Tofino switch.

D. Hardware Switch Performance

We conduct micro-benchmarking focusing on the processing latency calculated by the difference between timestamps at the entry of the first submodel and the exit points (i.e., the last recirculation). We utilize a Wedge100BF-32X switch, which contains an Intel Tofino packet processing ASIC; we emulate the multi-submodel inference testbed by sequentially deploying three submodels on the same switch, and calculating the processing delay by summing the packet-wise delays measured until each packet becomes exited.

Fig. 16 shows the processing latency of NNSplit and TINIEE with various configurations (i.e., model size and confidence threshold). For NNSplit, the processing latency increases as more layers are included in the model. This is because a higher number of neurons leads to more recirculations, which induces proportional hop latencies. Likewise, the processing delays of TINIEE steadily increase as the confidence threshold becomes higher, due to larger recirculations as the exit point moves backward. However, this increasing trend is not linear; interestingly, the increment remains relatively low until $c = 0.75$, leading to sufficiently low processing latency (e.g.,

TABLE II

NETWORK TRAFFIC DIFFERENCE WITH OPTIMAL (%): SOLVER VS ESP

Topology	Scheme	Confidence threshold					
		0.55	0.65	0.75	0.85	0.95	AVG
Italian	Solver	0.34	0.27	0.22	0.49	0.85	0.43
	ESP	0.02	0.03	0.06	0.37	0.92	0.28
NSFNET	Solver	4.88	3.62	2.64	3.15	3.74	3.61
	ESP	0.12	0.39	1.20	3.49	7.47	2.53

TABLE III

COMPUTATION TIME (S): OPTIMAL VS SOLVER VS ESP

Topology	Scheme	Confidence threshold					
		0.55	0.65	0.75	0.85	0.95	AVG
Italian	Optimal	50.2	50.8	51.5	52.1	52.4	51.4
	Solver	1.07	1.06	1.07	1.07	1.07	1.07
	ESP	0.82	0.94	1.05	1.20	1.26	1.05
NSFNET	Optimal	157	160	163	167	168	163
	Solver	2.41	2.42	2.43	2.42	2.43	2.42
	ESP	0.89	1.04	1.20	1.38	1.46	1.20

38.2% and 17.5% less latency than NNsplit with m_{123} and m_{12} when $c = 0.65$). Recalling that TINIEE achieves comparable inference performance for $c \geq 0.65$, this indicates that the effectiveness of the proposed adaptive in-network inference is also valid in the hardware switch.

E. ESP Algorithm Benchmarking

We compare the performance of the ESP algorithm in terms of the objective value (i.e., network traffic) and the computation time with the optimal solution and the MILP solver. To obtain the optimal solution, we used a brute-force method. In addition, the Gurobi optimizer [48] is utilized as a MILP solver. We perform the evaluation in the Italian and NSFNET topologies, with the number of nodes being 10 and 14, respectively. The results are measured using a server with an Intel i9-10900X CPU, featuring 10 cores and 64 GB of RAM. The results are arranged in Table II and Table III, and the best value in each case is highlighted in bold.

Table II shows the difference in network traffic between the solver and ESP based on the optimal solution. In Table II, the difference of ESP from the optimal solution gradually increases as the confidence threshold increases, although the solver performance does not exhibit a clear tendency. However, the difference in ESP is relatively low, except in the case where the confidence threshold is excessively large (e.g., 0.95), and the average value is lower than that of the solver in both topologies. Meanwhile, Table III presents the computation time of the three schemes. As shown in Table III, both the solver and ESP demonstrate much less computation time than the brute-force case. Moreover, in terms of scalability, ESP shows superior performance, as it has a lower increase in computation time (14.3%) compared to the solver (126%) and the brute-force method (217%) on average as the topology changes from Italian to NSFNET. In summary, ESP is near-optimal and time-efficient, demonstrating even better performance and scalability than the commercial MILP solver.

VI. RELATED WORK

In-Network Inference. There has been a proliferation of research implementing models in PDP (i.e., in-network inference) due to its outstanding adaptability and performance. Sanvito et al. [8] showed the possibility of using NIC as a neural network accelerator by deploying a convolutional neural network (CNN) for image classification. Zhong et al. [9] partially utilized PDP for inference in the optical domain by using programmable switches to perform a non-linear activation function, such as a rectified linear unit (ReLU). Qin et al. [10] implemented BNN in the software switch and SmartNIC for in-network intrusion detection. Xie et al. [11] proposed a scheme to transform NN into a decision tree using knowledge distillation to overcome the computational and resource constraints of the hardware switch. Siracusano et al. [12] implemented BNN in NetFPGA and SmartNIC to support in-network traffic analysis. Zhang et al. [13] utilized a pruning-based model compression method to reduce the memory and computational costs. Yann et al. [14] implemented an INI system using the binarized recurrent neural network (RNN), where the server continues if the output value is not sufficiently large. Zhao et al. [21] also devised a hardware-friendly RNN model to be implemented in the hardware P4 switch.

Although making significant improvements to implement lightweight but effective DNN models in PDP, however, the scope of these works are bounded in the small-scale environment with a few or single switch, where the probability and the impact of utilizing multiple switches are not considered.

Distributed In-Network Inference. As PDP programs become more complex and resource-consuming, current studies have investigated orchestrating the in-network functionality in a distributed fashion. Sultana et al. [42] proposed an automated framework that disaggregates and maps P4 programs considering heterogeneous target devices. Furthermore, Zheng et al. [43] proposed a distributed in-network computing framework that optimizes the placement of P4 program segmentations, taking into account segmentation overhead such as latency and resource consumption.

In this context, to alleviate the deployment and processing costs, several works have focused on distributing DNN models across multiple nodes. Saquetti et al. [15] distributed neurons in programmable switches and orchestrated them to minimize the maximum distance between adjacent layers. Razavi et al. [16] proposed a conceptual distributed in-network DNN system and analyzed challenges using the well-known DNN model, AlexNet. He et al. [17] conducted a case study in which a model is split and deployed in multiple user plane functions (UPFs) supporting PDP. Dao and Lee [18] proposed a mechanism in which a model is divided horizontally (i.e., across the entire model) into two pieces; after checking the output on the primary switch, it is decided whether to continue the inference. Li et al. [19] proposed a framework for splitting the DNN model and deploying it on programmable switches layer-by-layer to alleviate the burden per switch.

However, existing works do not consider the submodel placement problem in realistic networks where many placement options exist. Furthermore, these solutions cannot adaptively control the trade-off between classification performance and processing cost during in-network inference. To the best of our knowledge, TINIEE is the first work to support

TABLE IV

COMPARISONS BETWEEN THE PROPOSED SCHEME AND RELATED WORKS

Scheme	Model	Dist.	Adapt. Infer.	Placem.	Implem.
[10]	BNN	✗	✗	✗	SmartNIC
[14]	RNN	✗	✓	✗	Tofino
[18]	ANN	✓	✓	✗	BMv2
[19]	BNN	✓	✗	✗	BMv2
TINIEE	BNN	✓	✓	✓	Tofino

adaptive in-network inference while orchestrating submodels considering the inference characteristics. Note that comparisons between TINIEE and INI schemes are summarized in Table IV.

VII. CONCLUSION

In this work, we proposed TINIEE, a traffic-aware adaptive in-network inference scheme. First, we designed adaptive in-network inference by applying the early-exit method, where the confidence score of the intermediate result is evaluated, and the inference is terminated if it is sufficient. To generate a compact calculation rule, we devised the GRACE algorithm, which adjusts the granularity of the confidence score and expresses the entries using exact and range matching operations. We also proposed the practical ESP algorithm, considering the exit tendency according to the confidence threshold. Evaluation results demonstrate that TINIEE can significantly reduce network costs (e.g., network traffic, maximum link utilization, and FCT) while maintaining comparable classification performance under various topologies in the real world. In future work, a method to control the confidence threshold, considering the dynamic network state, will be studied. Furthermore, we are planning to integrate PDP-optimized neural architectures [14], [21] into the TINIEE framework and evaluate how adaptive in-network inference is effective in various INI architectures.

REFERENCES

- [1] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends," *IEEE Access*, vol. 9, pp. 87094–87155, 2021.
- [2] H. Lee, J. Lee, H. Kim, and S. Pack, "Straggler-aware in-network aggregation for accelerating distributed deep learning," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4198–4204, Nov. 2023.
- [3] T. P. A. Group. (Jun. 2020). *INT Dataplane Specification*. [Online]. Available: https://p4.org/p4-spec/docs/INT_v2_1.pdf
- [4] W. Jiang, H. Jiang, J. Wu, and Q. Chen, "Accelerating distributed cloud storage systems with in-network computing," *IEEE Netw.*, vol. 37, no. 4, pp. 64–70, Jul. 2023.
- [5] T. Swamy, A. Rucker, M. Shahbaz, I. Gaur, and K. Olukotun, "Taurus: A data plane architecture for per-packet ML," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, New York, NY, USA, Feb. 2022, pp. 1099–1114.
- [6] Y. Chang, W. Li, and Z. Yang, "Network intrusion detection based on random forest and support vector machine," in *Proc. IEEE Int. Conf. Comput. Sci. Eng. (CSE) IEEE Int. Conf. Embedded Ubiquitous Comput. (EUC)*, vol. 1, Guangzhou, China, Jul. 2017, pp. 635–638.
- [7] N. Ramakrishnan and T. Soni, "Network traffic prediction using recurrent neural networks," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2018, pp. 187–193.
- [8] D. Sanvito, G. Siracusano, and R. Bifulco, "Can the network be the AI accelerator?," in *Proc. Morning Workshop Netw. Comput.*, New York, NY, USA, Aug. 2018, pp. 20–25.
- [9] Z. Zhong et al., "IOI: In-network optical inference," in *Proc. ACM SIGCOMM Workshop Opt. Syst.*, Aug. 2021, pp. 18–22.
- [10] Q. Qin, K. Poularakis, K. K. Leung, and L. Tassiulas, "Line-speed and scalable intrusion detection at the network edge via federated learning," in *Proc. IFIP Netw. Conf. (Networking)*, Jun. 2020, pp. 352–360.
- [11] G. Xie, Q. Li, Y. Dong, G. Duan, Y. Jiang, and J. Duan, "Mousika: Enable general in-network intelligence in programmable switches by knowledge distillation," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 1938–1947.
- [12] G. Siracusano et al., "Re-architecting traffic analysis with neural network interface cards," in *Proc. USENIX NSDI*, Renton, USA, Apr. 2022, pp. 513–533.
- [13] X. Zhang, L. Cui, F. P. Tso, W. Li, and W. Jia, "IN3: A framework for in-network computation of neural networks in the programmable data plane," *IEEE Commun. Mag.*, vol. 62, no. 4, pp. 96–102, Apr. 2024.
- [14] J. Yan et al., "Brain-on-switch: Towards advanced intelligent network data plane via NN-driven traffic analysis at line-speed," in *Proc. USENIX NSDI*, Santa Clara, CA, USA, Apr. 2024, pp. 419–440.
- [15] M. Saquetti et al., "Toward in-network intelligence: Running distributed artificial neural networks in the data plane," *IEEE Commun. Lett.*, vol. 25, no. 11, pp. 3551–3555, Nov. 2021.
- [16] K. Razavi, G. Karlos, V. Nigade, M. Mühlhäuser, and L. Wang, "Distributed DNN serving in the network data plane," in *Proc. 5th Int. Workshop P4 Eur.*, Rome, Italy, Dec. 2022, pp. 67–70.
- [17] J. He, H. Wu, X. Xiao, R. Bassoli, and F. H. P. Fitzek, "Functional split of in-network deep learning for 6G: A feasibility study," *IEEE Wireless Commun.*, vol. 29, no. 5, pp. 36–42, Oct. 2022.
- [18] T.-N. Dao and H. Lee, "JointNIDS: Efficient joint traffic management for on-device network intrusion detection," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13254–13265, Dec. 2022.
- [19] D. C. Li, M. R. Maulana, and L.-D. Chou, "NNSplit-SØREN: Supporting the model implementation of large neural networks in a programmable data plane," *Comput. Netw.*, vol. 222, Feb. 2023, Art. no. 109537.
- [20] A. T.-J. Akem, B. Bütün, M. Gucciardo, and M. Fiore, "Henna: Hierarchical machine learning inference in programmable switches," in *Proc. 1st Int. Workshop Native Netw. Intell.*, Rome, Italy, Dec. 2022, pp. 1–7.
- [21] Z. Zhao, Z. Li, Z. Song, F. Zhang, and B. Chen, "RIDS: Towards advanced IDS via RNN model and programmable switches co-designed approaches," in *Proc. IEEE Conf. Comput. Commun.*, May 2024, pp. 591–600.
- [22] H. Kim, S. Yoon, C. Bae, S. Lee, and S. Pack, "TINIEE: Traffic-aware adaptive in-network intelligence via early-exit strategy," in *Proc. 21st Annu. IEEE Int. Conf. Sens., Commun., Netw. (SECON)*, Dec. 2024, pp. 10–18.
- [23] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, Cancun, Mexico, Dec. 2016, pp. 2464–2469.
- [24] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu, and F. Wei, "BERT loses patience: Fast and robust inference with early exit," in *Proc. NeurIPS*, Dec. 2020, pp. 1–12.
- [25] S. Laskaridis, A. Kouris, and N. D. Lane, "Adaptive inference through early-exit networks: Design, challenges and directions," in *Proc. 5th Int. Workshop Embedded Mobile Deep Learn.*, Jun. 2021, pp. 1–6.
- [26] (2026). *Software Optimization Guide for AMD Athlon™64 and Opteron™Processors*. [Online]. Available: <https://cr.yip.to/bib/2004/amd-25112.pdf>
- [27] (2026). *Intel Tofino Programmable Ethernet Switch ASIC*. [Online]. Available: <https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/tofino-series.html>
- [28] P. Bosshart et al., "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, Jul. 2014.
- [29] G. Bianchi, M. Bonola, A. Capone, and C. Cascone, "OpenState: Programming platform-independent stateful openflow applications inside the switch," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 2, pp. 44–51, Apr. 2014.
- [30] A. Sivaraman et al., "Packet transactions: high-level programming for line-rate switches," in *Proc. ACM SIGCOMM Conf.*, Aug. 2016, pp. 15–28.
- [31] H. Kim, C. Bae, S. Yoon, H. Kim, H. Ku, and S. Pack, "MALOI: Multi-task-aware low-overhead in-network inference using programmable switch," *IEEE Commun. Mag.*, vol. 64, no. 2, pp. 127–132, Feb. 2026.
- [32] (2026). *P4Runtime*. [Online]. Available: <https://github.com/p4lang/p4runtime>

- [33] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman Company, 1979.
- [34] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Math.*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [35] *Mininet*. Accessed: Aug. 15, 2024. [Online]. Available: <https://github.com/mininet/mininet>
- [36] *BMv2*. Accessed: Aug. 15, 2024. [Online]. Available: <https://github.com/p4lang/behavioral-model>
- [37] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, Rome, Italy, Feb. 2016, pp. 407–414.
- [38] A. Habibi Lashkari, G. Draper Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy*, Porto, Portugal, Sep. 2017, pp. 253–262.
- [39] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proc. USENIX Secur.*, San Diego, CA, USA, Aug. 2004.
- [40] *Pytorch*. Accessed: May 9, 2025. [Online]. Available: <https://pytorch.org>
- [41] A. T-J. Akem, B. Butun, M. Gucciardo, and M. Fiore, "Jewel: Resource-efficient joint packet and flow level inference in programmable switches," in *Proc. IEEE Conf. Comput. Commun.*, Vancouver, BC, Canada, May 2024, pp. 1631–1640.
- [42] N. Sultana et al., "Flightplan: Dataplane disaggregation and placement for P4 programs," in *Proc. USENIX NSDI*, Apr. 2021, pp. 571–592.
- [43] C. Zheng et al., "DINC: Toward distributed in-network computing," in *Proc. ACM CoNEXT*, vol. 1, Paris, France, Dec. 2023, pp. 1–25.
- [44] M. Alizadeh et al., "CONGA: Distributed congestion-aware load balancing for datacenters," in *Proc. ACM Conf. SIGCOMM*, New York, NY, USA, Aug. 2014, pp. 503–514.
- [45] G. Mantelet, A. Cassidy, C. Tremblay, D. V. Plant, P. Littlewood, and M. P. Bélanger, "Establishment of dynamic lightpaths in filterless optical networks," *J. Opt. Commun. Netw.*, vol. 5, no. 9, pp. 1057–1065, Sep. 2013.
- [46] H. Huang, S. Huang, S. Yin, M. Zhang, J. Zhang, and W. Gu, "Virtual network provisioning over space division multiplexed optical networks using few-mode fibers," *J. Opt. Commun. Netw.*, vol. 8, no. 10, pp. 726–733, Oct. 2016.
- [47] R. Casellas, R. Martínez, R. Muñoz, L. Liu, T. Tsuritani, and I. Morita, "Dynamic provisioning via a stateful PCE with instantiation capabilities in GMPLS-controlled flexi-grid DWDM networks," in *Proc. 39th Eur. Conf. Exhib. Opt. Commun. (ECOC)*, Sep. 2013, pp. 1–3.
- [48] *Gurobi Optimizer*. Accessed: Sep. 15, 2024. [Online]. Available: <http://www.gurobi.com>
- [49] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, Aug. 2017, pp. 1321–1330.



Seongyeon Yoon (Member, IEEE) received the B.S. degree from the University of Seoul, Seoul, South Korea, in 2022, and the M.S. degree from the School of Electrical Engineering, Korea University, Seoul, in 2024. She is currently with LG Uplus Corporation, Seoul. Her research interests include programmable data plane, in-network computing, and network intelligence.



Chanbin Bae (Member, IEEE) received the B.S. degree from Korea University, Seoul, South Korea, in 2022, where he is currently pursuing the integrated M.S. and Ph.D. degree with the School of Electrical Engineering. His research interests include programmable networking and in-network computing.



Sanghoon Lee (Member, IEEE) received the B.S. degree from Ajou University, Suwon, South Korea, in 2024, and the M.S. degree from the School of Electrical Engineering, Korea University, Seoul, South Korea, in 2026. He is currently with SK Telecom, Seoul. His research interests include programmable networking and in-network computing.



Sangheon Pack (Senior Member, IEEE) received the B.S. and Ph.D. degrees in computer engineering from Seoul National University, Seoul, South Korea, in 2000 and 2005, respectively. He joined as a Faculty Member of Korea University, Seoul, in 2007, where he is currently a Professor with the School of Electrical Engineering. His research interests include softwarized networking (SDN/NFV), 5G/6G mobile core networks, mobile edge computing/programmable data plane, and vehicular networking. He was a recipient of the IEEE/Institute of Electronics and Information Engineers (IEIE) Joint Award for IT Young Engineers Award in 2017, Korean Institute of Information Scientists and Engineers (KIISE) Young Information Scientist Award in 2017, Korean Institute of Communications and Information Sciences (KICS) Haedong Young Scholar Award in 2013, and the IEEE ComSoc APB Outstanding Young Researcher Award in 2009. He served as the TPC Vice-Chair for information systems of the IEEE WCNC 2020, the Track Chair for the IEEE VTC 2020-Fall/2010-Fall and the IEEE CCNC 2019, and the Publication Co-Chair for the IEEE INFOCOM 2014 and ACM MobiHoc 2015. He is an Editor of IEEE INTERNET OF THINGS (IoT) JOURNAL, IEEE NETWORK, and IEEE TRANSACTIONS ON CONSUMER ELECTRONICS (TCE).



Heewon Kim (Member, IEEE) received the B.S. and Ph.D. degrees from the School of Electrical Engineering, Korea University, Seoul, South Korea, in 2021 and 2026, respectively. He is currently a Post-Doctoral Researcher with the Research Institute of Information and Communication Technology (RICT), Korea University. His research interests include programmable data plane, in-network computing, network monitoring, network intelligence, and network automation.